

STOCKLIMP ENTERPRISE SYSTEM

DOCUMENTO TÉCNICO OFICIAL

GUÍA DE ARQUITECTURA Y FUNCIONAMIENTO

Manual Integral de Flujo de Datos, Controladores, Base de Datos y Componentes

Este documento proporciona una radiografía técnica exhaustiva del ecosistema **StockLimp**, cubriendo el ciclo de vida de pedidos, flujos de entrada/salida cliente-servidor, seguridad por roles, modelo relacional MySQL, controladores backend en Laravel y componentes frontend en Expo / React Native Web.

PROYECTO

StockLimp Full-Stack

STACK TECNOLÓGICO

React Native Web + Laravel
11/12 + MySQL

FECHA / VERSIÓN

Septiembre 2026 • v2.4
Estable

1. Resumen Ejecutivo y Topología Global

StockLimp es una plataforma de software integral diseñada para la gestión logística de aprovisionamiento de productos de limpieza industrial, control horario de operarios (fichajes con justificantes médicos/laborales), administración de solicitudes de vacaciones por días hábiles y trazabilidad de rutas hacia centros de trabajo asignados.

Frontend Multiplataforma

Construido sobre **Expo 54** y **React Native Web (React 19)** con Expo Router 6. Proporciona una interfaz unificada adaptativa (Web escritorio, tablet y móvil Android/iOS) con animaciones nativas y gráficos SVG vectoriales.

Backend RESTful API

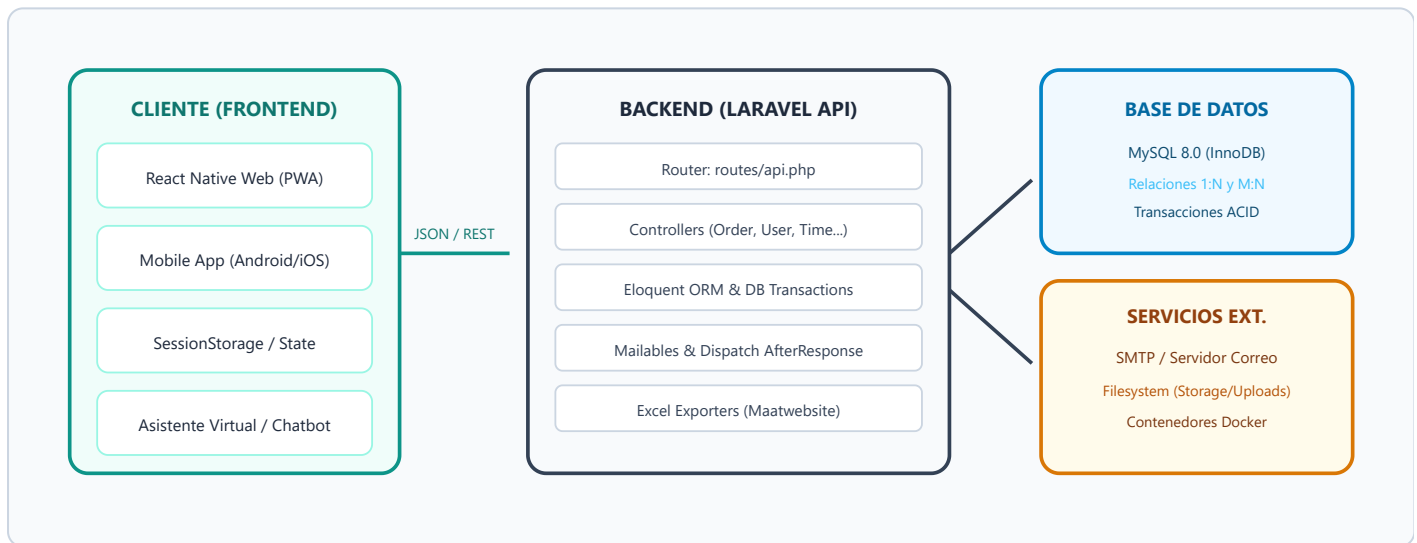
Desarrollado en **PHP 8.3 / Laravel**. Gestiona transacciones ACID concurrentes, políticas de autorización basadas en roles (RBAC), cola de envíos de correo asíncronos y generación de hojas de cálculo Excel XLSX.

Persistencia y Almacenamiento

Motor **MySQL 8.0 (InnoDB)** con integridad referencial estricta y llaves foráneas. Almacenamiento de fotos de perfil en Base64 procesadas en disco local y justificantes en el sistema de archivos de Laravel Storage.

Topología Arquitectónica

El siguiente diagrama ilustra la interacción entre los clientes, la capa API y los subsistemas de soporte:



2. Flujos de Datos de Entrada y Salida (E/S)

El sistema se rige por un esquema asíncrono cliente-servidor mediante peticiones HTTP estructuradas en formato JSON y multipart/form-data. A continuación se desglosan los ciclos de vida de datos fundamentales:

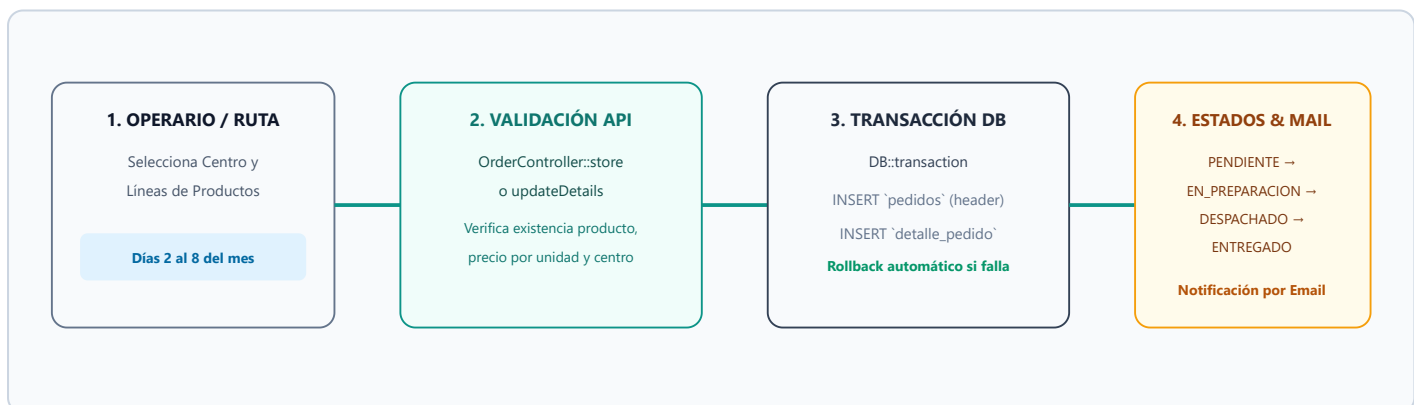
2.1. Ciclo de Autenticación y Recuperación de Acceso

Secuencia de Login y Verificación de Estado

- Entrada:** El operario introduce `email` y `password` en `HomeScreen`.
- Procesamiento Backend:** `UserController::login` busca al usuario por email. Comprueba el hash de contraseña utilizando `Hash::check()` para hashes Bcrypt (\$2y\$) y coincidencia directa para credenciales heredadas.
- Validación de Estado:**
 - Si el estado es `pendiente`, retorna error HTTP 403: "Tu cuenta está pendiente de aprobación por el Super Administrador".
 - Si el estado es `rechazado`, retorna error HTTP 403.
 - Si la contraseña es la temporal por defecto (12345), devuelve `debe_cambiar_password: true` bloqueando el acceso a módulos operativos hasta que ejecute el cambio obligatorio.
- Salida & Persistencia:** El backend devuelve el payload del usuario (ID, nombre, email, rol, ID de centro asignado). En navegador Web, el cliente lo graba en `sessionStorage.setItem('user')` manteniendo la sesión al recargar sin comprometer tokens en cookies inseguras.

2.2. Ciclo de Creación y Modificación de Pedidos

El flujo de pedidos incluye reglas de negocio estrictas temporales y transaccionales para evitar errores en inventario y rutas:



2.3. Ciclo de Control Horario (Fichajes y Justificantes)

- Consulta de Estado Inicial** (`POST /api/fichajes/actual`): Al abrir la pantalla de fichaje, se consulta el último registro sin cerrar (estado `trabajando` o `en_pausa`). Si existe, el cronómetro frontend sincroniza el tiempo transcurrido desde `hora_entrada`.
- Inicio de Turno** (`POST /api/fichajes/iniciar`): Registra fecha y hora exacta del servidor en UTC/local mediante Carbon, evitando discrepancias de hora del reloj del dispositivo cliente.
- Pausa/Reanudación** (`POST /api/fichajes/pausar`): Cambia entre los estados `trabajando` y `en_pausa` de forma inmediata.

- **Cierre con Documentos (`POST /api/fichajes/finalizar`):** Petición con soporte para subida de ficheros (`multipart/form-data`). Guarda el fichero en `storage/app/public/justificantes` y vincula la URL pública al registro del fichaje junto con observaciones.

2.4. Ciclo de Solicitud y Cómputo de Vacaciones

El sistema incluye un motor algorítmico en `VacacionController::calcularDiasLaborables` que itera día por día entre la fecha inicial y la fecha final, evaluando `$current->isWeekday()` para descartar sábados y domingos. Los operarios cuentan con un cupo anual de 22 días laborables; si la suma de solicitudes aprobadas o pendientes excede el límite, la API rechaza la solicitud de forma preventiva.

3. Controladores y Servicios Principales (Backend)

El backend implementa controladores delgados y cohesivos ubicados en `back/app/Http/Controllers`, apoyados en modelos Eloquent y servicios de exportación y mensajería:

Controlador	Endpoints Clave	Responsabilidad y Lógica Interna
UserController	<code>/usuarios/login</code> <code>/usuarios/aprobar</code> <code>/usuarios/rechazar</code> <code>/usuarios/update-role</code> <code>/usuarios/update-profile</code> <code>/usuarios/enviar-restablecimiento</code>	Controla autenticación, altas por invitación con estado <code>pendiente</code>, notificaciones de bienvenida y rechazo por email, actualización de perfil con almacenamiento de fotos en Base64 decodificado a JPG/PNG en <code>public/uploads/profiles</code>. <i>Protección de Seguridad:</i> Previene la eliminación o degradación de rol del último <code>super_admin</code> activo del sistema.
OrderController	<code>/pedidos</code> <code>/pedidos/multiple</code> <code>/pedidos/update-status</code> <code>/pedidos/update-multiple</code> <code>/pedidos/update-details</code> <code>/pedidos/exportar</code>	Gestiona la cabecera y el detalle de pedidos. Emplea <code>DB::transaction()</code> para garantizar atomicidad. Bloquea modificaciones a usuarios estándar fuera del intervalo del 2 al 8 del mes. <i>Email Dispatcher:</i> Al transicionar un pedido a estados como <code>DESPACHADO</code> o <code>ENTREGADO</code>, ejecuta <code>dispatch(...)->afterResponse()</code> para enviar <code>OrderStatusUpdated</code> por correo al operario sin congelar la respuesta HTTP.
ProductController	<code>/productos</code> <code>/productos/update</code> <code>/productos/delete</code> <code>/productos/exportar</code>	Catálogo maestro de insumos químicos y productos de limpieza. Controla SKU único, precios unitarios, stock actual, banderas de toxicidad (<code>es_toxico</code>) y generación de archivo Excel XLSX mediante <code>ProductExport</code>.
CenterController	<code>/centros_trabajo</code> <code>/centros_trabajo/update</code> <code>/centros_trabajo/delete</code>	Administración de centros físicos de trabajo organizados por número de ruta logística (1 a 17) y ciudad. <i>Auto-Sanación:</i> Su constructor evalúa mediante <code>Schema::hasColumn</code> si existen los campos <code>ciudad</code>, <code>numero_ruta</code> y <code>fecha_registro</code>, ejecutando migraciones silenciosas si faltan en base de datos.
TimeTrackingController	<code>/fichajes/actual</code> <code>/fichajes/iniciar</code> <code>/fichajes/pausar</code> <code>/fichajes/finalizar</code>	Máquina de estados de presencia horaria (<code>trabajando</code> ↔ <code>en_pausa</code> → <code>finalizado</code>). Almacena estampas de tiempo con Carbon y documentos anexos justificativos en <code>public/justificantes</code>.
VacacionController	<code>/vacaciones/disponibles</code> <code>/vacaciones</code> <code>/vacaciones/mis-vacaciones</code> <code>/vacaciones/todas</code> <code>/vacaciones/{id}/estado</code> <code>/vacaciones/{id}/cancelar</code>	Motor de cálculo de días laborables. Valida disponibilidad respecto al tope legal anual (22 días). Notifica por email a todos los administradores cuando un empleado emite una solicitud. Gestiona solicitudes de cancelación de vacaciones ya aprobadas.

Servicios de Notificación por Correo Electrónico (Mailables)

UserRegistered & UserApproved

Envía las credenciales iniciales (contraseña temporal 12345) y comunica al empleado la activación formal de su cuenta para iniciar labores.

OrderStatusUpdated

Notifica en tiempo real al operario cuando el personal de almacén despacha o entrega el material solicitado para su centro de trabajo.

PasswordResetMail

Remite una clave alfanumérica segura generada por el administrador cuando el usuario reporta olvido de credenciales.

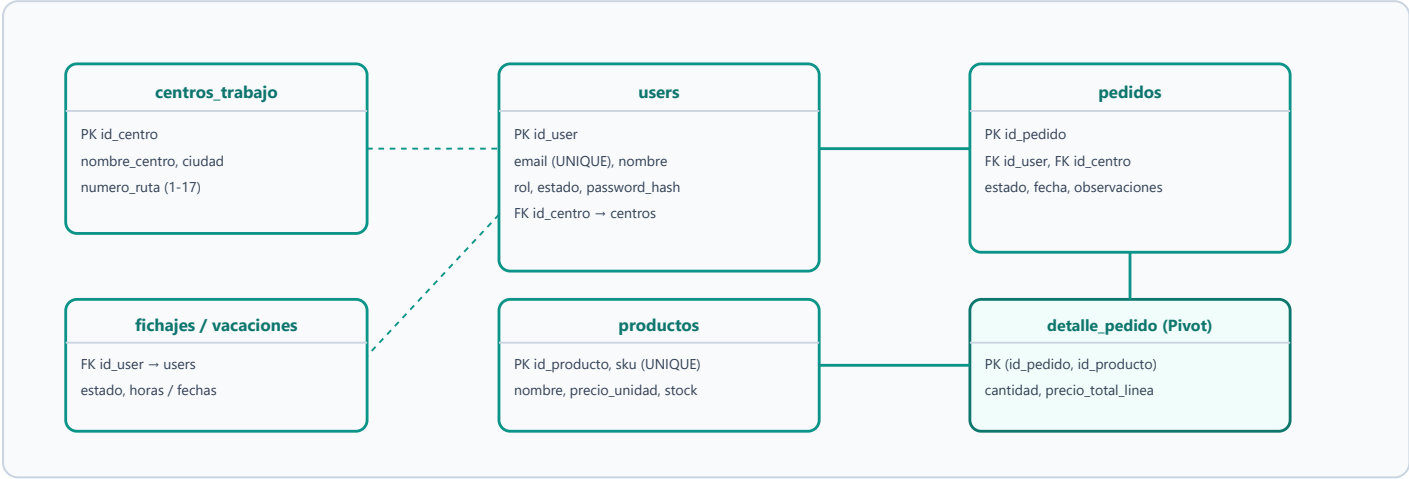
VacationRequestedMail

Emite una alerta a los supervisores indicando el rango de fechas solicitado y los comentarios adjuntos para su aprobación o denegación.

4. Manejo de Base de Datos y Modelo Relacional

El esquema de persistencia se almacena bajo el motor relacional **MySQL / MariaDB** con charset `utf8mb4` y collation `utf8mb4_general_ci`. A continuación se expone la estructura de las tablas maestras, transaccionales y pivote:

Diagrama Entidad-Relación Conceptual



Diccionario de Tablas Principales

Tabla	Campos Principales	Restricciones y Claves Foráneas
users	id_user, nombre, apellido, email, password_hash, rol, id_centro, estado, foto_perfil	PK: id_user. UNIQUE: email. FK: id_centro → centros_trabajo.id_centro (ON DELETE SET NULL). Roles válidos: super_admin, admin, usuario, repartidor.
centros_trabajo	id_centro, nombre_centro, direccion, ciudad, numero_ruta, telefono, contacto	PK: id_centro. numero_ruta restringido entre 1 y 17 para optimización logística de reparto.
productos	id_producto, nombre, sku, es_toxico, precio_unidad, stock_actual, imagen	PK: id_producto. UNIQUE: sku. precio_unidad DECIMAL(10,2), es_toxico TINYINT(1).
pedidos	id_pedido, id_user, id_centro, estado, fecha_creacion, observaciones	PK: id_pedido. FK: id_user → users.id_user, id_centro → centros_trabajo.id_centro. Estados: PENDIENTE, EN_PREPARACION, DESPACHADO, ENTREGADO, CANCELADO.
detalle_pedido	id_pedido, id_producto, cantidad_solicitada, precio_total_linea	PK Compuesta: (id_pedido, id_producto). FKs con ON DELETE CASCADE hacia pedidos y ON UPDATE CASCADE hacia productos.
fichajes	id, id_user, estado, hora_entrada, hora_salida, comentarios, documento_path	PK: id. FK: id_user → users.id_user (CASCADE). Estados: trabajando, en_pausa, finalizado.
vacaciones	id_vacacion, id_user, fecha_inicio, fecha_fin, estado, comentarios	PK: id_vacacion. FK: id_user → users.id_user (CASCADE). Estados: pendiente, aprobada, rechazada, cancelada, solicita_cancelacion.

5. Componentes Frontend y Manejo de Estado

El frontend implementa una arquitectura basada en componentes desacoplados con **Expo Router** y persistencia en **SessionStorage**. A continuación se analiza cada componente clave:

HomeScreen (src/app/index.tsx) • Orquestador y Auth Guard

Es la vista raíz que gestiona el enrutamiento interno. Si no hay sesión, despliega el panel de Login y el botón flotante del **Asistente Virtual (Chatbot inteligente)**. Tras autenticar al usuario, renderiza el Sidebar de navegación y monta dinámicamente la pestaña adecuada según el rol.

Roles Administrativos (super_admin / admin):

Acceso completo a Centros, Productos, Pedidos, Panel de Control, Aprobación de Usuarios, Vacaciones y Analítica.

Rol Empleado / Operario (usuario):

Acceso limitado exclusivamente a Pedidos (sólo suyos y en días 2-8), Control Horario (Fichaje + Vacaciones) y Perfil.

Desglose de Vistas Principales (Dashboard Views)

1. OrdersView (src/components/dashboard/OrdersView.tsx)

El módulo más extenso de la aplicación. Implementa:

- **Filtros por Operario y Rutas:** Búsqueda en tiempo real con animación de barra expansible y filtro por número de ruta.
- **Acciones Masivas:** Casillas de verificación para marcar múltiples pedidos y cambiar su estado en lote (por ejemplo, a **ENTREGADO**) con confirmación mediante modal personalizado.
- **Ventana Temporal Restringida:** Modal de advertencia que impide a operarios modificar pedidos fuera de los días 2 al 8 de cada mes, con botón de bypass para pruebas de administración.
- **Exportación XLSX:** Descarga directa del libro de pedidos formateado con cabecera y desglose de líneas.

2. TimeTrackingView (src/components/dashboard/TimeTrackingView.tsx)

Gestiona la jornada laboral diaria del operario:

- **Cronómetro SVG Circular:** Dibuja anillos de progreso vectoriales interactivos calculando horas y minutos trabajados respecto a la jornada diana (8 horas).
- **Subida de Justificantes:** Selector de documentos (PDF o imágenes) mediante **expo-document-picker** para adjuntar partes de baja médica o justificaciones de retraso.
- **Calendario de Vacaciones:** Modal con **react-native-calendars** configurado en español. Permite seleccionar fecha inicio y fecha fin, calculando al vuelo los días hábiles restantes.

3. AnalyticsView (src/components/dashboard/AnalyticsView.tsx)

Motor de inteligencia de negocio para almacén y compras:

- **Agregación Dinámica:** Cruza los pedidos con sus detalles y productos sumando unidades consumidas en el año o mes seleccionado.
- **Gráficos Vectoriales Propios:** Gráfico de donut/pastel SVG con porcentajes de cuota por producto y gráfico de barras comparativo mensual (2025 vs 2026).
- **Resiliencia Offline:** Incluye dataset histórico simulado (**MOCK_MONTHLY_DATA**) para visualización inmediata si el servidor API está inaccesible.

4. ControlPanelView.tsx

Permite a los administradores gestionar el listado completo de operarios, cambiar roles en caliente, vincular usuarios a centros de trabajo y detonar el restablecimiento de contraseñas por correo.

5. VacationsAdminView.tsx

Bandeja de entrada donde el departamento de RRHH o Administración aprueba, rechaza o cancela las solicitudes de vacaciones emitidas por los operarios de plantilla.

6. ApproveUsersView.tsx

Flujo de admisión de personal: interfaz doble donde los administradores invitan nuevos usuarios y el Super Admin autoriza o rechaza los registros pendientes.

7. ProfileView.tsx

Edición de información personal y carga de foto de perfil con recorte/conversión a Base64 y validación de tamaño máximo en cliente (5 MB).

6. Seguridad, Infraestructura y Despliegue

6.1. Matriz de Permisos por Rol (RBAC)

Módulo / Funcionalidad	super_admin	admin	repartidor	usuario (operario)
Ver y gestionar todos los pedidos	SÍ	SÍ	SÍ (Por Ruta)	NO (Sólo Propios)
Modificar pedidos fuera de días 2 al 8	SÍ (Bypass)	SÍ (Bypass)	NO	NO (Bloqueado)
Crear y editar catálogo de productos	SÍ	SÍ	NO	NO
Aprobar altas de nuevos usuarios	SÍ (Exclusivo)	NO	NO	NO
Aprobar / Denegar vacaciones	SÍ	SÍ	NO	NO
Fichaje diario y pedir vacaciones propias	SÍ	SÍ	SÍ	SÍ

6.2. Infraestructura Docker Compose y Cloud Render

El proyecto se encuentra totalmente contenedorizado para despliegues locales e integración continua:

Definición de Servicios en docker-compose.yml

- `stocklimp-db` : Imagen `mysql:8.0` en puerto 3306 con volumen persistente `dbdata` .
- `stocklimp-back` : Contenedor PHP/Laravel compilado vía Dockerfile propio, ejecutándose en el puerto 8000 y enlazado con la red de base de datos.
- `stocklimp-front` : Contenedor Node.js que ejecuta Expo Web en el puerto 8081 con soporte para polling de cambios en caliente (`CHOKIDAR_USEPOLLING=true`).

Despliegue en la Nube (render.yaml)

Configurado con dos servicios principales: `stocklimp-backend` como Web Service en entorno PHP / Docker y `stocklimp-db` como base de datos administrada. El frontend en producción redirige sus peticiones automáticamente al endpoint seguro HTTPS `https://stocklimp-backend.onrender.com/api` cuando no se encuentra en modo `__DEV__` .

6.3. Guía Rápida de Comandos para Mantenimiento

```
# 1. Iniciar el Backend Laravel localmente:
cd back
php artisan serve --port=8000

# 2. Iniciar el Frontend Expo en modo Web:
cd front
npm run web

# 3. Levantar todo el stack con Docker:
docker-compose up -d --build

# 4. Exportar o restaurar la Base de Datos:
mysql -u root stocklimp < "Base de datos/bbdd.sql"
```